

Web Application Vulnerability Assessment Report

SQL Injection and Cross-Site Scripting Assessment Using Burp Suite and DVWA

Parameter Details

Parameter	Details
Prepared By	Rudresha RK
Target Application	Damn Vulnerable Web Application (DVWA)
Target URL	http://localhost/dvwa
Assessment Platform	Kali Linux
Browser	Mozilla Firefox
Intercepting Proxy	Burp Suite Community Edition
Web Server	Apache2
Database	MySQL
Assessment Type	Web Application Vulnerability Assessment
Vulnerabilities Tested	SQL Injection, Reflected Cross-Site Scripting
Date	June 2026
Institution	InternSpark Internship
Classification	CONFIDENTIAL / INTERNAL ONLY

1. Introduction

1.1 Purpose

This report presents the findings of a Web Application Vulnerability Assessment conducted against the Damn Vulnerable Web Application (DVWA) within a controlled laboratory environment. The assessment was performed as part of the cybersecurity internship program to understand common web application vulnerabilities and to gain practical experience using professional web security testing tools.

The primary objective of the assessment was to identify security weaknesses related to SQL Injection and Cross-Site Scripting (XSS), analyze how these vulnerabilities can be exploited, and recommend appropriate mitigation strategies based on industry best practices. Burp Suite Community Edition was used as the primary interception and testing tool to capture, inspect, modify, and replay HTTP requests between the client browser and the web application.

All testing activities were performed only against DVWA, an intentionally vulnerable application designed for cybersecurity education. No unauthorized systems or production environments were targeted during this assessment.

1.2 Scope

The assessment was conducted exclusively within an isolated local testing environment running on Kali Linux. The target application was DVWA hosted on the localhost web server. The security level of DVWA was configured to "Low" in order to demonstrate common web application vulnerabilities without defensive protections.

The scope of testing included the following modules:

- SQL Injection
- Reflected Cross-Site Scripting (XSS)
- HTTP Request Interception
- Request Modification using Burp Suite Repeater

The assessment did not include testing of external systems, production environments, authentication bypass outside the designated SQL Injection module, Remote Code Execution, File Upload vulnerabilities, or Command Injection.

1.3 Objectives

The objectives of this assessment were:

- Understand the architecture and workflow of a web application.
 - Configure Burp Suite as an HTTP interception proxy.
 - Capture HTTP requests exchanged between the browser and DVWA.
 - Analyze request parameters submitted by the application.
 - Identify SQL Injection vulnerabilities.
 - Identify Reflected Cross-Site Scripting vulnerabilities.
 - Validate identified vulnerabilities through manual payload injection.
 - Understand the security risks associated with insecure input validation.
 - Recommend mitigation strategies based on OWASP security guidelines.
 - Produce a professional vulnerability assessment report suitable for technical review and internship documentation.
-

2. Executive Summary

A comprehensive web application security assessment was performed against DVWA using Burp Suite Community Edition. The assessment successfully identified multiple vulnerabilities associated with improper input validation and insecure server-side processing.

The SQL Injection module was found to be vulnerable to multiple injection techniques, including authentication bypass and UNION-based SQL Injection. These vulnerabilities demonstrated that user-controlled input was directly incorporated into backend SQL queries without adequate validation or parameterization.

Additionally, the Reflected Cross-Site Scripting module was found to execute arbitrary JavaScript supplied by the user, indicating insufficient output encoding and input sanitization.

Burp Suite successfully intercepted all HTTP requests exchanged between the browser and the application, allowing manual modification of parameters before forwarding requests to the server. Burp Repeater was used extensively to replay modified requests and validate the existence of identified vulnerabilities.

The assessment demonstrates how improper validation of user-supplied input can expose web applications to serious security risks, including unauthorized database access, disclosure of sensitive information, execution of malicious client-side scripts, session theft, and potential compromise of application integrity.

Vulnerability Distribution Matrix

Severity	Count
Critical	2
High	1
Medium	1
Low	0
Total	4

3. Assessment Environment

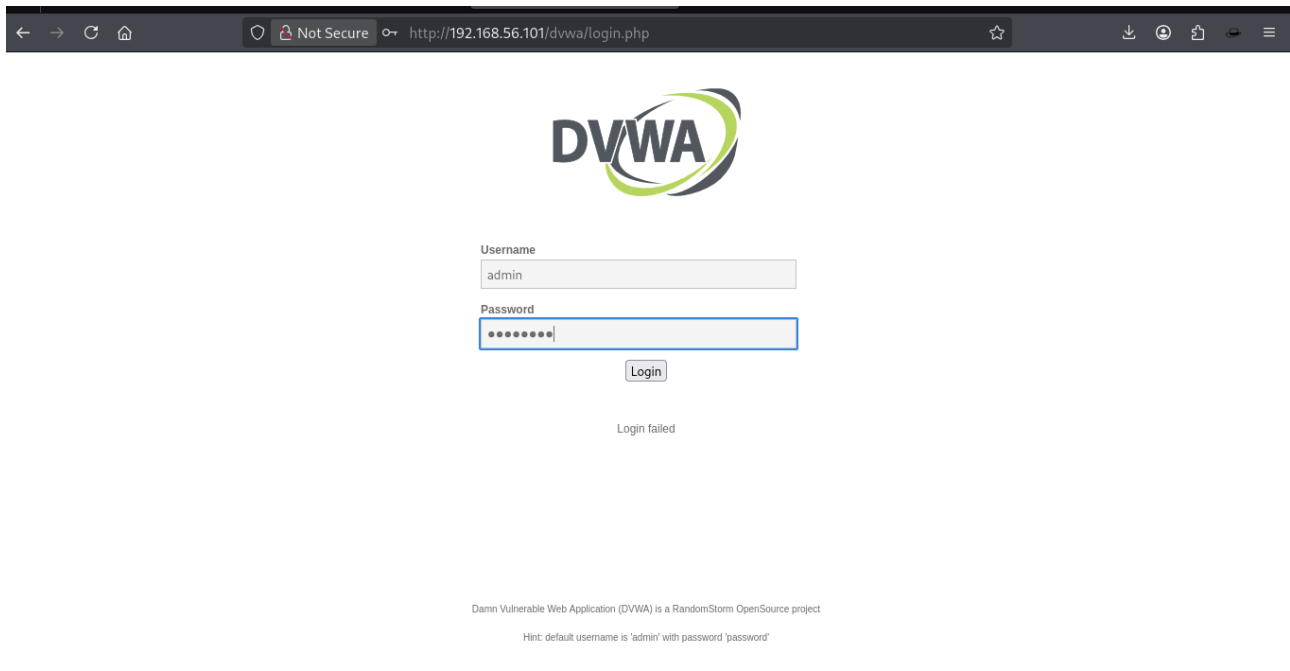
Component	Configuration
Operating System	Kali Linux
Browser	Mozilla Firefox
Web Application	Damn Vulnerable Web Application (DVWA)
Proxy Tool	Burp Suite Community Edition
Database	MySQL
Web Server	Apache2
PHP Version	PHP
Security Level	Low

4. Assessment Methodology

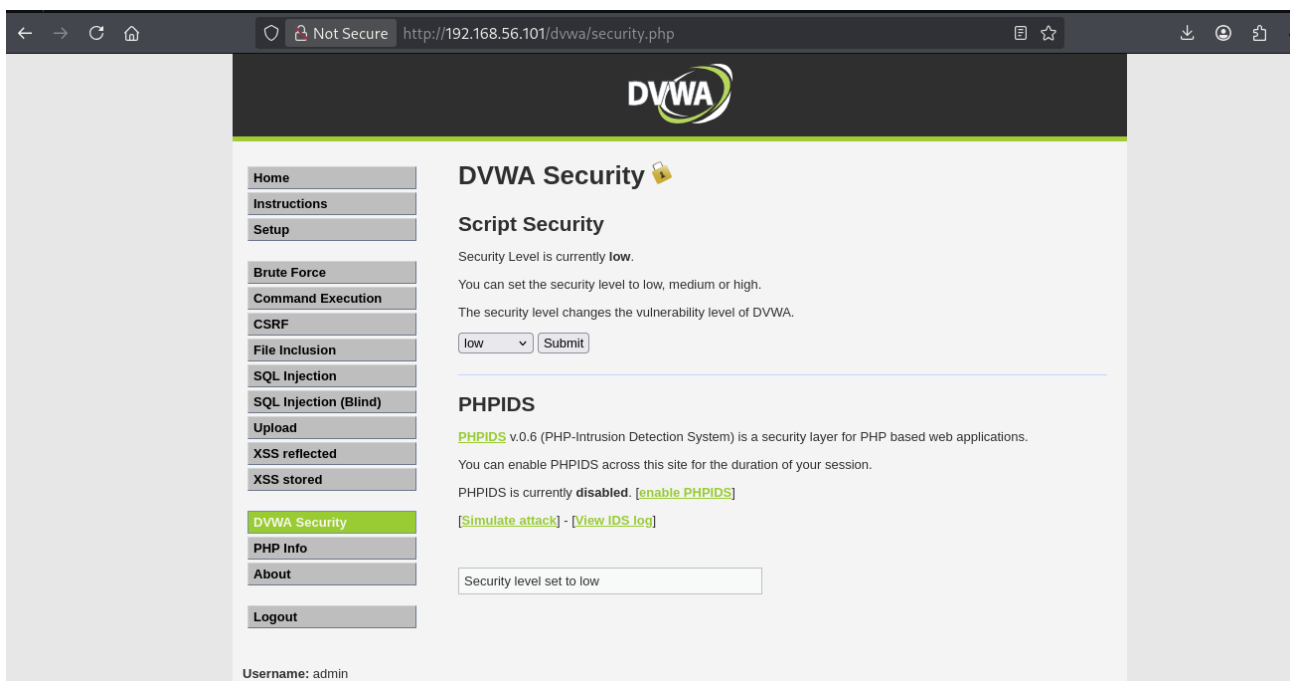
The assessment followed a structured methodology similar to industry-standard web application penetration testing procedures.

Step 1 – Environment Setup

DVWA was installed and configured on the local Apache server. MySQL services were initialized successfully, and the DVWA database was created using the setup utility. The application security level was configured to "Low" to allow vulnerability demonstration.



The screenshot shows the DVWA login page in a web browser. The URL is `http://192.168.56.101/dvwa/login.php`. The page features the DVWA logo at the top. Below the logo, there are input fields for "Username" (containing "admin") and "Password" (masked with dots). A "Login" button is positioned below the password field. Below the button, the text "Login failed" is displayed. At the bottom of the page, a small note states: "Damn Vulnerable Web Application (DVWA) is a RandomStorm OpenSource project" and "Hint: default username is 'admin' with password 'password'".

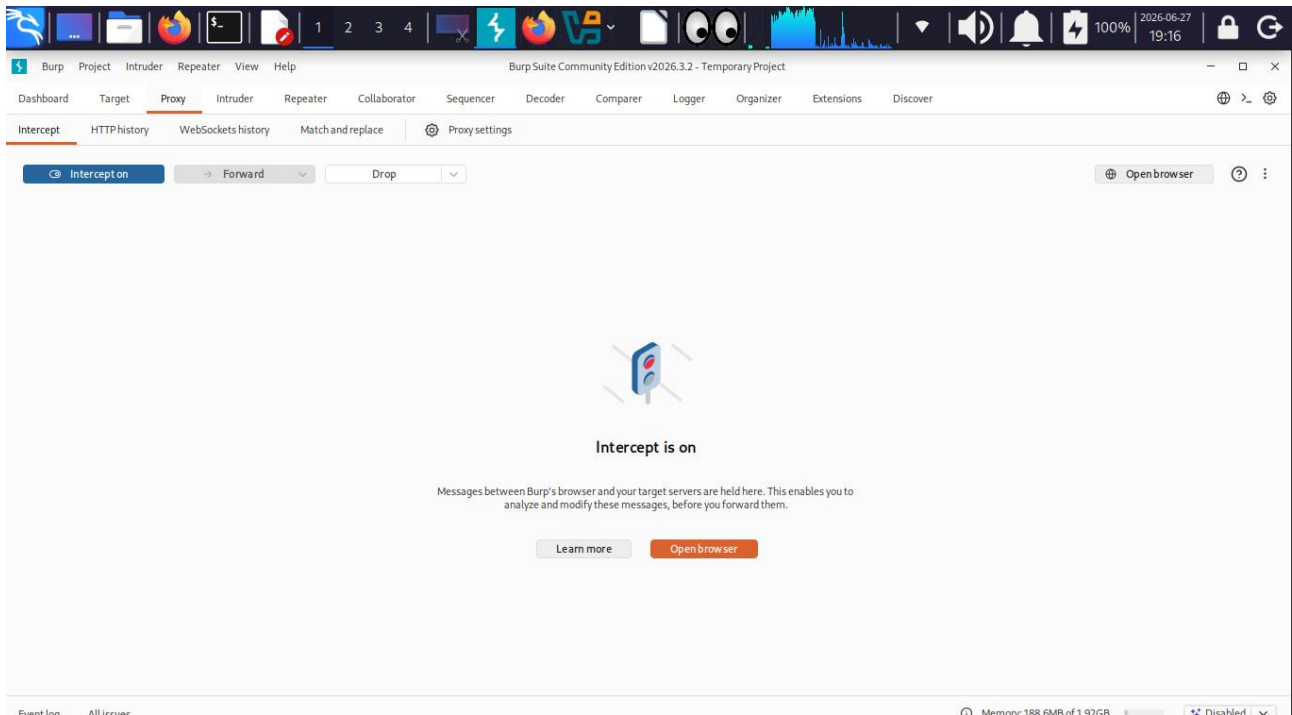


The screenshot shows the DVWA Security page in a web browser. The URL is `http://192.168.56.101/dvwa/security.php`. The page has a dark header with the DVWA logo. On the left side, there is a sidebar menu with various options: Home, Instructions, Setup, Brute Force, Command Execution, CSRF, File Inclusion, SQL Injection, SQL Injection (Blind), Upload, XSS reflected, XSS stored, DVWA Security (highlighted), PHP Info, About, and Logout. The main content area is titled "DVWA Security" and includes a "Script Security" section. It states "Security Level is currently low." and "You can set the security level to low, medium or high." Below this, there is a dropdown menu set to "low" and a "Submit" button. The "PHPIDS" section follows, stating "PHPIDS v.0.6 (PHP-Intrusion Detection System) is a security layer for PHP based web applications." and "You can enable PHPIDS across this site for the duration of your session." It also shows "PHPIDS is currently disabled" with links to "enable PHPIDS", "Simulate attack", and "View IDS log". At the bottom, a text box displays "Security level set to low".

Step 2 – Burp Suite Configuration

Burp Suite Community Edition was launched and configured as an intercepting proxy. Firefox browser traffic was routed through Burp Suite using the default proxy configuration (127.0.0.1:8080).

Proxy Intercept was enabled to capture all HTTP requests generated during interaction with DVWA.

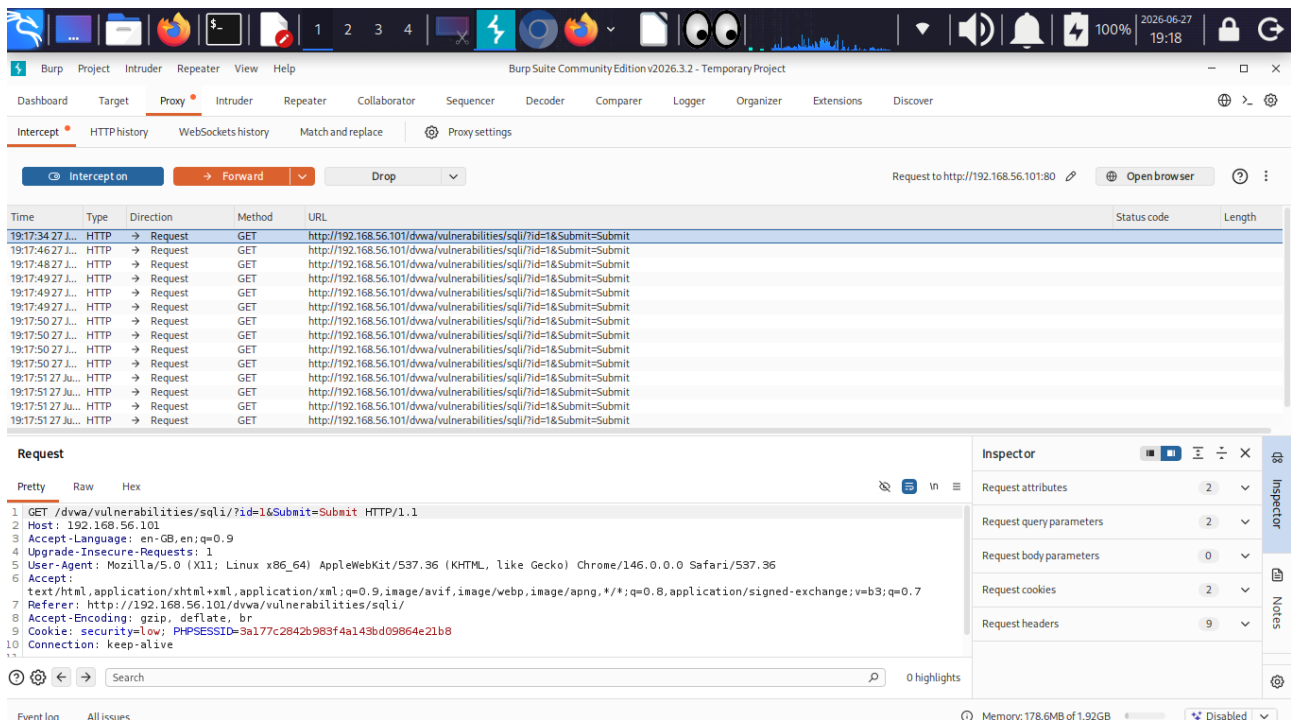


Step 3 – Request Interception

HTTP requests generated by DVWA were intercepted before reaching the web server.

Captured requests were inspected to identify parameters submitted by the application. Parameters such as "id" within the SQL Injection module and "name" within the XSS module were selected for further testing.

Each captured request was forwarded to Burp Repeater for manual modification.



Step 4 – Payload Injection

Multiple payloads were manually inserted into intercepted HTTP requests using Burp Repeater.

SQL Injection payloads included:

- 1
- 1'#
- 1' OR '1'='1
- 1' UNION SELECT user,password FROM users#

Reflected XSS payload included:

Each modified request was replayed against the application to evaluate the server response.

Step 5 – Response Analysis

Server responses were analyzed to determine whether the application correctly validated user input.

The SQL Injection payloads successfully manipulated backend SQL queries, while the XSS payload executed within the client's browser.

These responses confirmed the existence of exploitable vulnerabilities within the application.

Step 6 – Documentation

All successful tests were documented using screenshots captured during the assessment. Each vulnerability was analyzed, assigned a severity rating, and accompanied by recommended mitigation strategies based on OWASP best practices.

The following sections provide detailed technical findings for each identified vulnerability.

5. Detailed Technical Findings

Finding F-01 — Basic SQL Injection Vulnerability

Field	Details
Severity	HIGH
Vulnerability Type	SQL Injection
OWASP Category	A03:2021 – Injection
CWE Identifier	CWE-89 – Improper Neutralization of Special Elements used in an SQL Command
CVSS v3.1	8.8 (High)
Assessment Module	DVWA → SQL Injection

Description

The SQL Injection module of DVWA was tested to determine whether user-supplied input was properly validated before being processed by the backend MySQL database. Burp Suite Community Edition was configured as an intercepting proxy to capture the HTTP request generated when a User ID was submitted through the application.

Initially, the application was tested using a normal input value of 1. The intercepted request was forwarded to the server without modification, and the application returned the corresponding user record from the database. This confirmed that the parameter was directly used to retrieve information from the backend database.

The request was then forwarded to Burp Repeater to allow manual manipulation of the "id" parameter. The response indicated that the application did not implement any server-side input validation or parameterized SQL queries, making the input field vulnerable to SQL Injection attacks.

Evidence

HTTP Request Parameter

id=1

HTTP Method

GET

Testing Tool

Burp Suite Repeater

Application Response

The application returned the user information associated with User ID 1 without performing any input validation or sanitization.

The screenshot shows the Burp Suite Repeater interface. The 'Request' tab is active, displaying a GET request to `http://192.168.56.101/dvwa/vulnerabilities/sqli/?id=1&Submit=Submit`. The 'Response' tab is also active, showing the server's response. The response is an HTTP 200 OK status, indicating a successful request. The response body contains HTML content, including a title 'Damn Vulnerable Web App (DVWA) v1.0.7 :: Vulnerability: SQL Injection' and a link to 'More info'. The 'Inspector' panel on the right shows the request attributes, including the method (GET) and the path (`/dvwa/vulnerabilities/sqli/`).

This screenshot shows the same Burp Suite Repeater interface, but with a different request and response. The 'Request' tab displays a GET request to `http://192.168.56.101/dvwa/vulnerabilities/sqli/?id=16&Submit=Submit`. The 'Response' tab shows the server's response, which includes a form with a 'Submit' button and a 'More info' link. The 'Inspector' panel on the right shows the request attributes, including the method (GET) and the path (`/dvwa/vulnerabilities/sqli/`). The 'Selected text' panel on the right shows the selected text from the response, which is the 'More info' link.

Risk / Impact

The successful processing of unsanitized SQL parameters demonstrates that the application directly interacts with backend SQL statements using user-controlled input.

If deployed in a production environment, this weakness could allow attackers to:

- Enumerate database records
- Retrieve confidential customer information
- Discover table structures
- Perform authentication bypass attacks
- Modify application data
- Delete database records
- Execute administrative SQL commands

The vulnerability significantly increases the attack surface of the application and may ultimately lead to complete compromise of sensitive organizational data.

Root Cause

The application directly concatenates user input into SQL statements without using prepared statements or parameterized queries.

No server-side validation mechanisms are implemented to verify whether supplied input contains SQL control characters.

Remediation Steps

1. Replace dynamic SQL queries with parameterized prepared statements.
 2. Validate all incoming parameters using strict allow-list validation.
 3. Escape special SQL characters where necessary.
 4. Apply least privilege principles to database accounts.
 5. Disable detailed SQL error messages.
 6. Deploy a Web Application Firewall (WAF).
 7. Conduct regular penetration testing and code reviews.
-

Finding F-02 — SQL Injection Authentication Bypass

Field	Details
Severity	CRITICAL
Vulnerability Type	Authentication Bypass via SQL Injection
OWASP Category	A03:2021 – Injection
CWE Identifier	CWE-89
CVSS v3.1	9.8 (Critical)
Assessment Module	DVWA → SQL Injection

Description

Following confirmation of SQL Injection within the application, additional payloads were tested using Burp Suite Repeater to determine the overall impact of the vulnerability.

The original parameter value

```
1
```

was replaced with the payload

```
1' OR '1'='1
```

This payload manipulates the SQL WHERE clause by inserting an always-true logical expression.

Instead of searching for a single user record, the database evaluated the injected expression as TRUE, causing the SQL query to return all available records.

The successful execution of this payload confirmed that the application failed to distinguish between legitimate user input and executable SQL commands.

Payload Used

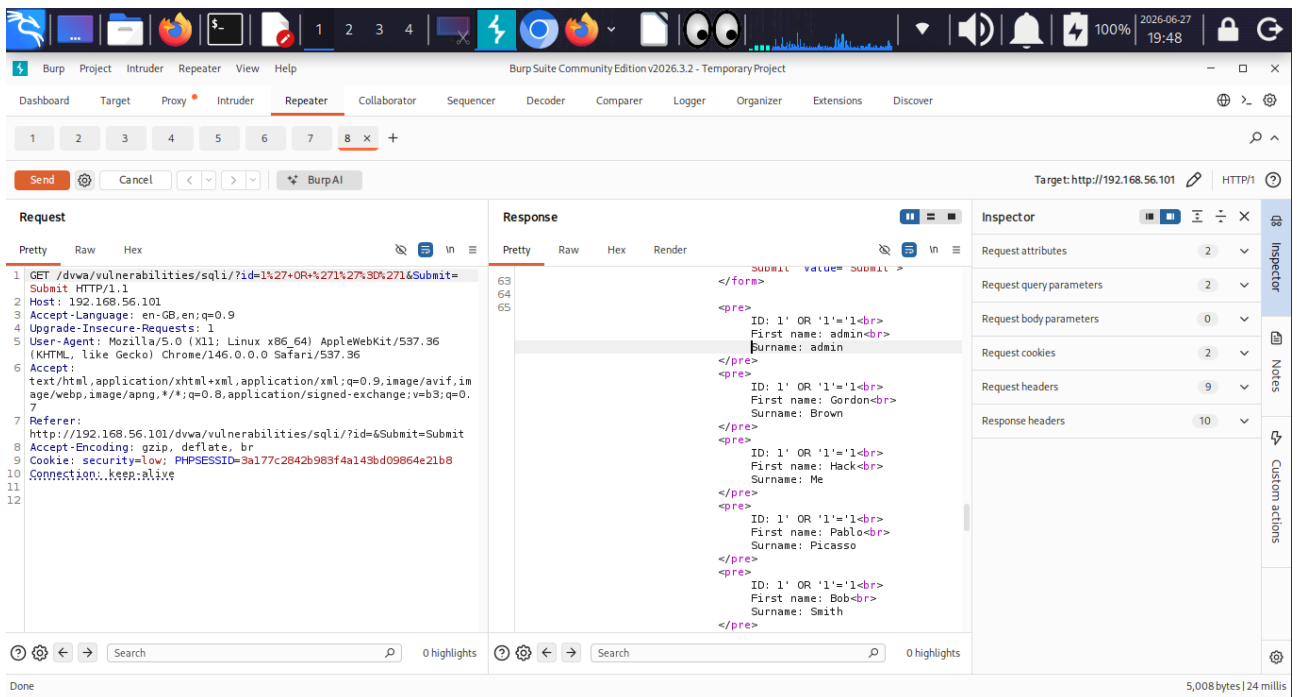
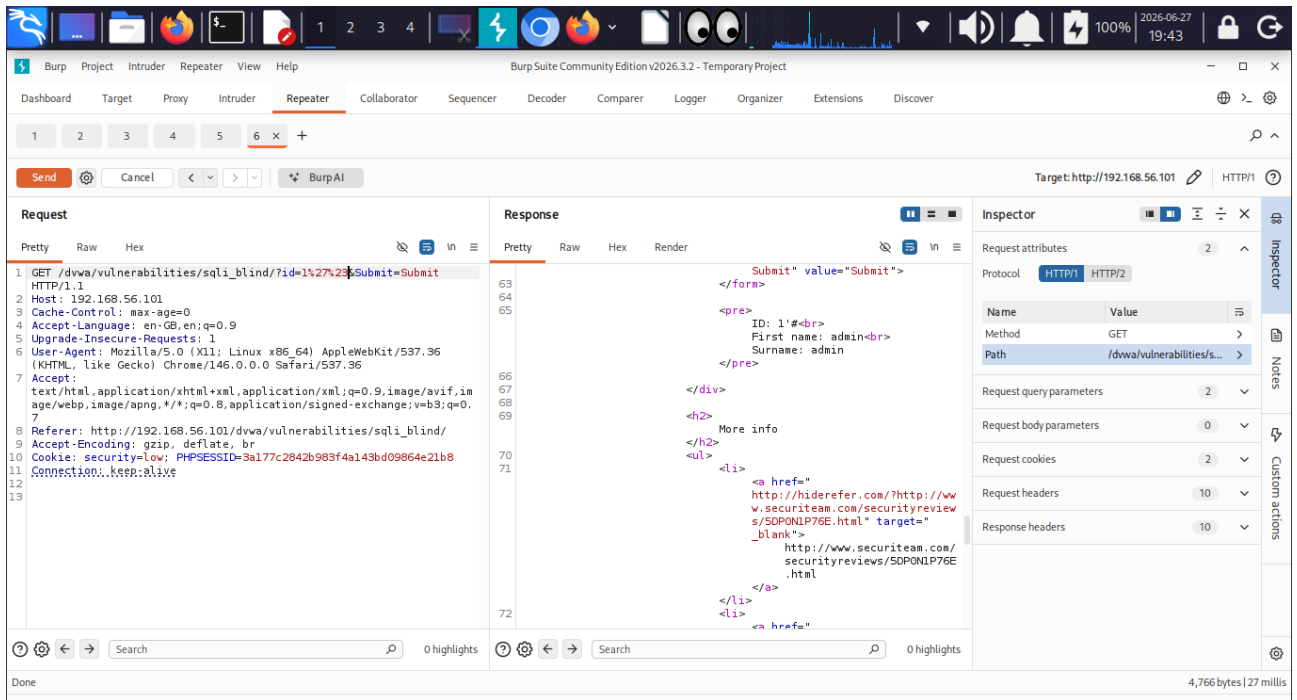
```
1' OR '1'='1
```

Evidence

The manipulated request was successfully processed by the application.

Instead of displaying a single user entry, DVWA returned multiple database records.

This demonstrates that arbitrary SQL syntax supplied by the user was interpreted as part of the backend SQL query.



Risk / Impact

Authentication bypass vulnerabilities allow attackers to ignore application logic entirely.

An attacker may:

- Access protected application areas
- Retrieve confidential information
- Enumerate application users

- Obtain sensitive records
- Circumvent intended access controls

If this vulnerability existed within a production authentication system, unauthorized administrative access could potentially be achieved.

Root Cause

Application logic fails to distinguish between executable SQL syntax and user input.

User-controlled parameters are concatenated directly into SQL queries without parameterization.

Remediation Steps

1. Implement parameterized SQL queries.
 2. Never concatenate user input into SQL statements.
 3. Validate numeric fields before processing.
 4. Enforce secure authentication mechanisms.
 5. Deploy server-side input validation.
 6. Monitor SQL logs for abnormal query patterns.
-

Finding F-03 — UNION-Based SQL Injection

Field	Details
Severity	CRITICAL
Vulnerability Type	UNION SQL Injection
OWASP Category	A03:2021 – Injection
CWE Identifier	CWE-89
CVSS v3.1	9.8 (Critical)
Assessment Module	DVWA → SQL Injection

Description

To evaluate whether sensitive database information could be extracted, UNION-based SQL Injection testing was performed.

The following payload was injected into the "id" parameter:

```
1' UNION SELECT user,password FROM users#
```

The UNION operator combines the results of two SQL queries into a single output.

Because the application failed to validate the supplied parameter, the backend database executed both queries successfully.

The application subsequently returned usernames together with password hashes stored inside the users table.

This confirmed that arbitrary database information could be extracted through SQL Injection.

Payload Used

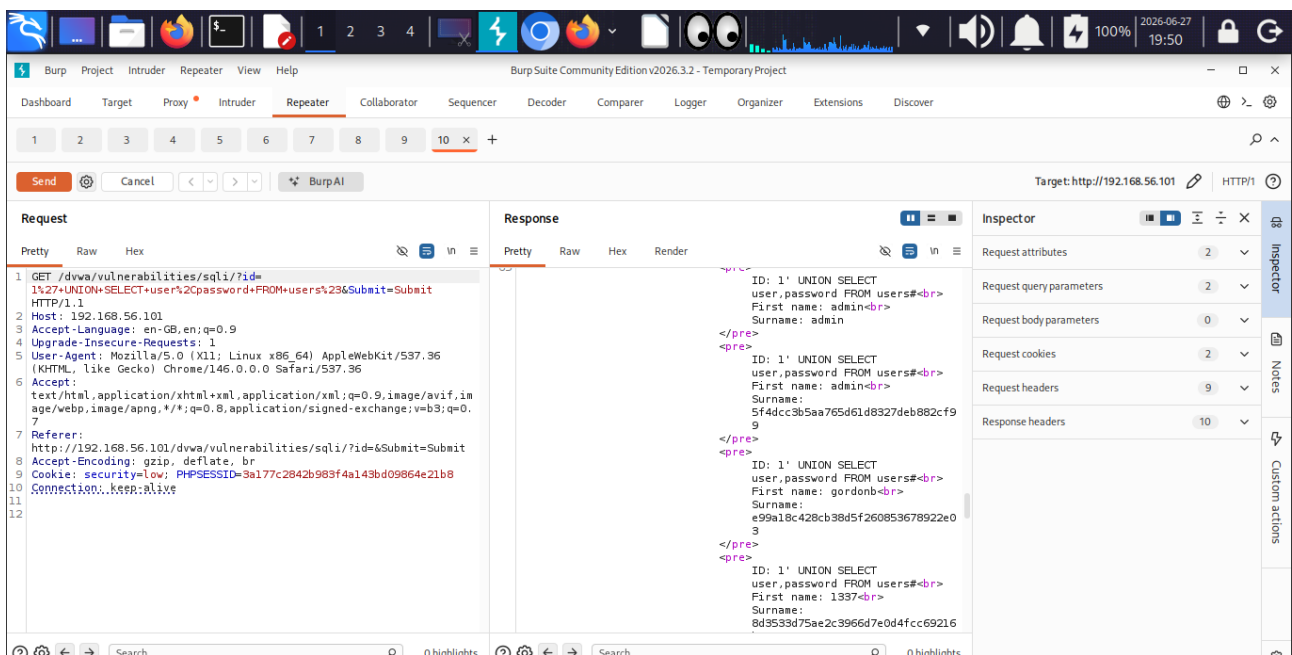
```
1' UNION SELECT user,password FROM users#
```

Evidence

The application displayed usernames together with corresponding password hashes extracted directly from the backend MySQL database.

Sensitive information was disclosed without authentication.

This confirms that the SQL Injection vulnerability can be exploited to perform database enumeration and information disclosure attacks.



Risk / Impact

Successful UNION-based SQL Injection may allow attackers to:

- Enumerate database tables
- Extract usernames
- Retrieve password hashes
- Discover administrative accounts
- Obtain confidential business information

- Perform privilege escalation
- Facilitate further attacks against user accounts

This represents one of the most severe categories of web application vulnerabilities.

Root Cause

Improper handling of user-controlled SQL parameters.

Absence of prepared statements.

Lack of server-side validation.

No SQL parameterization.

Remediation Steps

1. Replace dynamic SQL queries with prepared statements.
2. Disable verbose database error reporting.
3. Validate all parameters before processing.
4. Apply least privilege database permissions.
5. Encrypt sensitive information.
6. Conduct periodic secure code reviews.
7. Implement Web Application Firewall protection.

Finding F-04 — Reflected Cross-Site Scripting (XSS)

Field	Details
Severity	HIGH
Vulnerability Type	Reflected Cross-Site Scripting (XSS)
OWASP Category	A03:2021 – Injection
CWE Identifier	CWE-79 – Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting')
CVSS v3.1	8.2 (High)
Assessment Module	DVWA → XSS (Reflected)

Description

The Reflected Cross-Site Scripting (XSS) module of DVWA was evaluated to determine whether the application properly validated and encoded user-supplied input before displaying it within the browser. Cross-Site Scripting is one of the most common web application vulnerabilities and occurs

when an application includes untrusted user input in a web page without appropriate validation or output encoding.

During the assessment, Burp Suite Community Edition was configured as an intercepting proxy to capture the HTTP request generated when data was submitted through the Reflected XSS input field. The intercepted request was analyzed using Burp Suite Proxy and forwarded to Burp Repeater to facilitate manual payload modification.

Initially, a normal text string was submitted to understand the application's expected behavior. After confirming normal functionality, a JavaScript payload was injected into the "name" parameter to determine whether the application would execute client-side script supplied by the user.

The payload used during testing was:

```
<script>alert("hacked")</script>
```

The modified request was forwarded to the web server using Burp Repeater. Upon receiving the server response, the browser immediately executed the injected JavaScript code and displayed an alert dialog containing the text "hacked". The successful execution of arbitrary JavaScript confirmed that the application reflected user-controlled input directly into the HTML response without performing server-side sanitization or output encoding.

The assessment verified that the application is vulnerable to Reflected Cross-Site Scripting.

Payload Used

```
<script>alert("hacked")</script>
```

Testing Procedure

The following methodology was used to verify the existence of the vulnerability:

1. Logged into DVWA using administrator credentials.
2. Configured DVWA Security Level to **Low**.
3. Navigated to the **Reflected XSS** module.
4. Enabled Burp Suite Proxy Intercept.
5. Submitted a normal request through the application.
6. Captured the HTTP request using Burp Suite Proxy.
7. Forwarded the intercepted request to Burp Repeater.
8. Replaced the original parameter value with the JavaScript payload.
9. Sent the modified request to the application.
10. Observed the browser response and confirmed execution of the injected JavaScript.

The successful execution of the payload demonstrated that the application failed to sanitize user-controlled input before including it in the generated HTML page.

Evidence

Testing Tool:

Burp Suite Community Edition

Testing Module:

DVWA → Reflected XSS

Injected Payload:

```
<script>alert("hacked")</script>
```

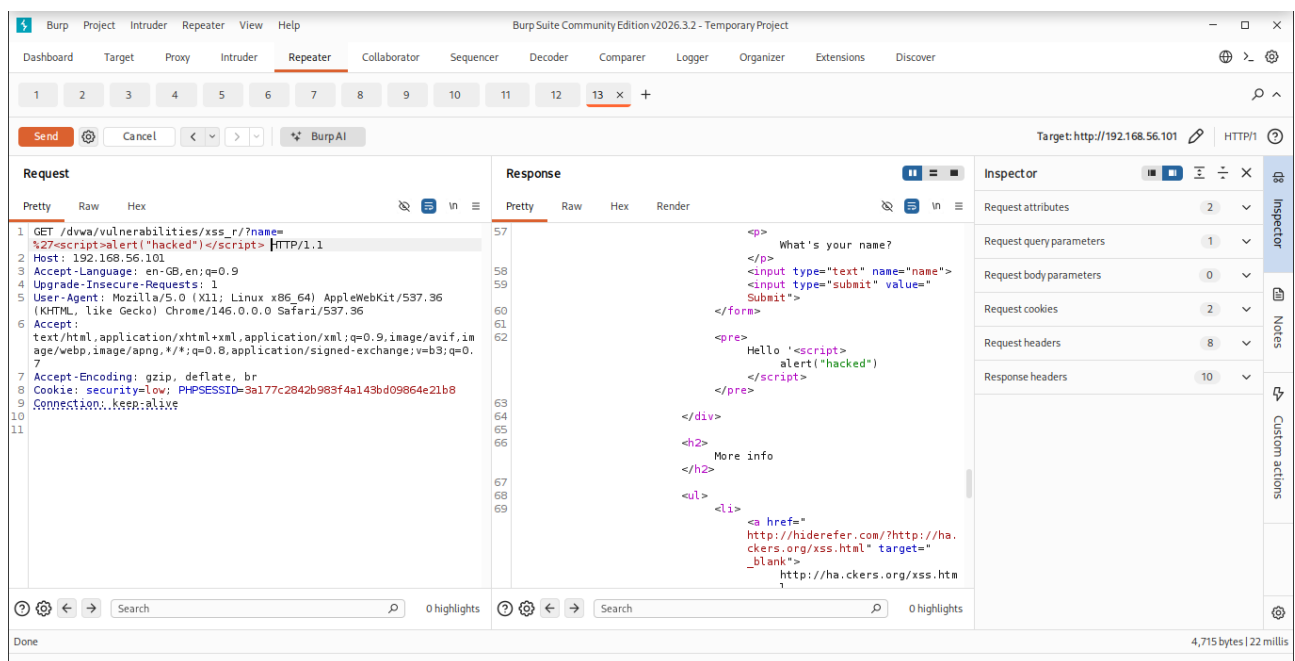
Application Response:

The browser displayed a JavaScript alert box containing the message:

hacked

The alert box appeared immediately after the modified request was processed by the application.

This confirms that the server reflected the supplied input directly into the generated HTML page without escaping special characters.



Technical Analysis

Cross-Site Scripting occurs because the application directly reflects user-controlled input into the HTML response without validating or encoding special characters.

Normally, HTML special characters such as:

<
>
"
'
&

should be converted into safe HTML entities before being displayed to the browser.

Instead, the application returned the payload exactly as supplied by the user.

Consequently, the browser interpreted the input as executable JavaScript instead of plain text.

This behavior allows attackers to execute arbitrary scripts within the security context of the vulnerable website.

Risk / Impact

Reflected Cross-Site Scripting vulnerabilities present significant security risks because malicious JavaScript executes within the victim's browser while appearing to originate from the trusted application.

If exploited against a production environment, attackers could:

- Steal active session cookies
- Hijack authenticated user sessions
- Capture usernames and passwords
- Redirect users to malicious phishing websites
- Modify webpage content
- Display fraudulent login forms
- Perform actions on behalf of authenticated users
- Execute malicious JavaScript code
- Harvest sensitive user information
- Damage application reputation

Because the attack executes within the victim's browser, traditional network security controls often cannot detect or prevent exploitation.

Root Cause

The application directly reflects user-controlled input into dynamically generated HTML pages.

The application does not perform:

- Input validation

- Output encoding
- HTML escaping
- JavaScript context encoding

As a result, malicious HTML and JavaScript supplied by the attacker become part of the generated webpage and execute automatically when rendered by the browser.

Security Implications

The identified vulnerability demonstrates inadequate implementation of secure coding practices.

Failure to properly sanitize user input increases the likelihood of:

- Client-side code execution
- Session hijacking
- Identity theft
- Credential disclosure
- Browser-based malware delivery
- Cross-site request forgery facilitation
- Loss of user trust

Although DVWA intentionally contains this weakness for educational purposes, identical vulnerabilities continue to appear in real-world web applications and remain one of the most frequently exploited security issues.

Remediation Steps

1. Validate all user input on the server side before processing.
2. Apply context-aware output encoding before displaying user-controlled data.
3. Convert HTML special characters into safe HTML entities.
4. Implement a strong Content Security Policy (CSP) to reduce the impact of injected scripts.
5. Enable the HttpOnly flag on session cookies to reduce the risk of session theft.
6. Implement Secure and SameSite cookie attributes.
7. Avoid dynamically inserting user-controlled data into HTML, JavaScript, CSS, or DOM elements without appropriate encoding.
8. Utilize modern web security libraries that automatically encode untrusted input.
9. Perform regular secure code reviews and penetration testing.
10. Follow the OWASP Cross-Site Scripting Prevention Cheat Sheet during application development.

Severity Justification

The Reflected Cross-Site Scripting vulnerability enables attackers to execute arbitrary JavaScript within a victim's browser. Successful exploitation may lead to session hijacking, credential theft, phishing attacks, unauthorized actions performed on behalf of authenticated users, and compromise of sensitive client-side information.

Although exploitation generally requires user interaction, the potential impact on confidentiality and application integrity is significant. Based on the observed behavior during testing, this vulnerability has been assigned a **High** severity rating.

Assessment Result

Status: Vulnerability Confirmed

The assessment successfully confirmed that the Reflected XSS module of DVWA executes user-supplied JavaScript without performing adequate input validation or output encoding. The injected payload executed successfully within the browser, demonstrating the presence of a Reflected Cross-Site Scripting vulnerability. Appropriate mitigation strategies have been recommended to eliminate this weakness in production environments.